SENTINEL X

Enterprise Zero-Trust Biometric Security Platform

PROJECT BLUEPRINT & TECHNICAL DOCUMENTATION

System plan • ERD • DFD • Architecture • API design • Security model • Testing • Roadmap

Version 1.0 | 27 October 2026 | Planning document

Document Control

This blueprint defines a portfolio/hackathon-ready prototype. It is a design baseline, not a claim that the system is implemented, production-certified, or compliant with a regulation.

Field	Details
Project	SENTINEL X — Enterprise Zero-Trust Biometric Security Platform
Purpose	Project proposal, system analysis, implementation guide, and security test plan
Users	Employee, Security Administrator, Auditor, protected application
Proposed stack	React + TypeScript; Python FastAPI; PostgreSQL; optional Redis; WebAuthn/passkeys; Docker
Biometric privacy	Use platform authenticators; do not store raw fingerprint images or face templates for passkey authentication.
Version/status	1.0 — Planning baseline; validate requirements and design during implementation

Contents

• 1. Executive summary and goals

• 2. Scope, actors, and requirements

• 3. System architecture

• 4. Context diagram and DFD Level 0 / Level 1

• 5. ERD and data dictionary

• 6. Authentication and access workflows

• 7. API design

• 8. Security, privacy, and threat model

• 9. UI screens and user stories

• 10. Test plan and acceptance criteria

• 11. Implementation roadmap and repository structure

• 12. Deployment, demo plan, risks, and references

1. Executive Summary

SENTINEL X is a web-based zero-trust access gateway for protected applications and documents. It uses WebAuthn/FIDO2 passkeys where available, server-side authorization, session controls, audit trails, and explainable risk rules. A browser asks a supported authenticator to verify the user; the server validates a cryptographic response and decides whether access is allowed.

Project vision

Let users access protected web applications from supported devices and browsers using the authentication methods available to them, while giving administrators visibility and control over access decisions.

Primary goals

• Provide cross-device passkey authentication using standards-based WebAuthn.

• Enforce authorization on the server for every protected resource and sensitive action.

• Support role-based access control (RBAC), step-up authentication, session revocation, and account recovery.

• Record security events and show explainable risk signals in an administrator dashboard.

• Demonstrate tamper-evident audit logs and repeatable defensive security tests.

• Package the project so reviewers can run it using Docker and seeded demo accounts.

Important capability boundary

A website cannot force every device to provide a fingerprint or iris scan. WebAuthn can request user verification through a supported platform or roaming authenticator, which may use a fingerprint, face recognition, device PIN, or another local method. Genuine iris recognition needs specialized support and is not universally exposed by browsers. Provide secure fallback and recovery flows.

Success measures

Measure	Prototype target
Authentication	Passkey registration and login work on at least two supported device/browser combinations.
Authorization	Unauthorized API/resource requests are denied server-side, including direct URL/API attempts.
Monitoring	Authentication, policy decisions, revocations, and admin actions appear in the audit view.
Security tests	Automated tests cover challenges, access control, session expiry, rate limits, and event integrity.
Reproducibility	A reviewer can launch the stack using documented Docker instructions and run the demo checklist.

2. Scope, Actors, and Requirements

In scope for the first release

• User accounts, passkey enrollment/removal, login, logout, recovery, and session management.

• A protected document/resource portal to prove that the gateway enforces access restrictions.

• Roles: Security Administrator, Employee, Auditor; optional Resource Owner later.

• Access policies based on identity, role, resource sensitivity, authentication freshness, and simple risk signals.

• Security event dashboard, filters, event details, alert acknowledgement, and emergency session revocation.

• Rule-based risk scoring with reasons; consider anomaly detection only after establishing safe test data.

• Dockerized local demo and CI tests.

Out of scope for MVP

• Guaranteed biometric compatibility with every device, browser, or authenticator.

• Native iris recognition without specialized imaging hardware and a separately validated component.

• Replacing a company's identity provider, endpoint detection, SIEM, or enterprise IAM product.

• Autonomous AI blocking without review, broad network scanning, or testing third-party systems.

• Storing raw biometric data or building custom fingerprint matching.

Actors and permissions

Actor	Responsibilities	Restrictions
Employee	Enroll/manage own passkeys; sign in; access permitted resources; view own sessions.	Cannot change roles/policies or view other users' sensitive data.
Security Administrator	Manage users, roles, policies, sessions, and security alerts.	Privileged actions must be strongly authenticated, authorized, and logged.
Auditor	Read approved audit events and reports.	Read-only; no policy or account changes.
Protected application	Ask gateway to authorize a resource/action.	Must validate server-side authorization; frontend hiding is not security.
Authenticator	Verify user locally and produce a cryptographic assertion.	Private key stays protected by authenticator/provider; app does not receive raw biometric.

Functional requirements

ID	Requirement	Priority
FR-01	Register/authenticate using WebAuthn; verify challenge, origin, RP ID, signature, credential status, and required user verification.	Must
FR-02	Manage users, roles, permissions, and protected resources.	Must
FR-03	Create, validate, expire, and revoke sessions securely.	Must
FR-04	Enforce access policy on every protected API/resource request.	Must
FR-05	Log authentication, authorization, policy, and administrative events.	Must
FR-06	Display risk score and reason codes for suspicious demo events.	Should
FR-07	Allow authorized emergency session revocation and account suspension.	Should
FR-08	Export filtered audit reports without exposing secrets.	Could

3. System Architecture

Use a modular monolith for the first release. Keep clear boundaries between identity, policy, audit, and resource modules; split services only if there is a measured need.

Layer	Components and responsibility
Client	React + TypeScript web app; browser WebAuthn API; employee portal; admin dashboard.
Transport	HTTPS/reverse proxy; secure headers; rate limiting at edge where available.
Application — FastAPI	Authentication service; policy engine; session service; protected-resource API; audit/event service; risk rules; admin API.
Data — PostgreSQL	Users, credential metadata, roles, permissions, sessions, resources, policies, events, alerts.
Short-lived state — optional Redis	Single-use WebAuthn challenges, rate-limit counters, optional cache/queue.
Delivery	Docker; CI pipeline; migrations; configuration/secrets; backups and documented demo data.

Recommended technology choices

Area	Choice	Notes
Frontend	React + TypeScript + Vite	Accessible UI; never trust client-side role checks.
Backend	Python + FastAPI	Central policy checks, typed schemas, input validation.
Authentication	Maintained WebAuthn/FIDO2 library	Do not write cryptographic protocol logic from scratch.
Database	PostgreSQL	Migrations, foreign keys, least-privilege DB role, backups.
Short-lived state	Redis optional	Short-lived challenges and rate limits; DB remains source of truth for permissions.
Testing	Pytest + API tests + Playwright	Positive and negative authorization tests; browser compatibility matrix.
Deployment	Docker Compose; HTTPS for hosted demo	Keep secrets out of Git; use environment variables/secrets.

Design principles

• Deny by default; grant only the minimum permissions required.

• Enforce authorization in the backend for every resource and action.

• Use a maintained WebAuthn library; validate challenge, origin, RP ID, signature, and user-verification requirements.

• Protect session cookies with Secure, HttpOnly, and appropriate SameSite settings; rotate session IDs after authentication and privilege changes.

• Use short-lived, single-use challenges and rate-limit sensitive endpoints.

• Keep biometric matching local to the authenticator whenever using passkeys; do not collect biometric templates.

4. Context Diagram and Data Flow Diagrams

Context diagram — SENTINEL X as one system

External entity	Input to SENTINEL X	Output from SENTINEL X
Employee / browser	Login request, passkey assertion, resource request	Authentication result, access decision, session status
Security Administrator	Policy changes, revoke requests, report queries	Alerts, audit views, administrative result
Protected application	Resource/action authorization request	Allow/deny decision and correlation ID
Authenticator / platform	User-verified cryptographic assertion	Browser-mediated verification prompt; no raw biometric sent to app

DFD Level 0 — major processes and data stores

Process / entity	Data flow	Store / destination
Employee / browser	Passkey options, assertion, login result, resource request	P1 Authentication; P2 Session/access gateway
P1 Authentication	Credential lookup and verified assertion outcome	D1 Users & passkey credentials
P2 Policy engine	User/role/resource/context → allow/deny and reason codes	D2 Roles, permissions, policies, resources
P3 Event & risk engine	Security event → risk score, alert, audit record	D3 Security events and alerts
P4 Admin / audit interface	Admin action, report query, revocation command	D1, D2, D3, session store

DFD Level 1 — passkey login and resource access

Step	Process / flow	Security control
1	Browser requests one-time authentication challenge.	Rate limit; cryptographically random challenge with short expiry.
2	Browser invokes WebAuthn authenticator; user verifies locally.	Authenticator handles local verification; do not collect biometric data.
3	Browser sends assertion to backend.	Validate challenge, origin, RP ID, signature, credential status, and required user verification.
4	Backend creates protected session.	Rotate session identifier; secure cookie; expiry and revocation support.
5	Client requests protected resource.	Check session, role, resource permission, risk policy, and authentication freshness.
6	Backend returns allow/deny and records decision.	Deny by default; log reason code and correlation ID; do not log secrets.
7	Admin views event or revokes session.	Admin strong authentication, permission checks, and audit of admin actions.

5. Entity Relationship Diagram (ERD)

Logical ERD — suggested relational model. Final constraints and indexes should be reviewed alongside migrations and application flows.

Entity	Key fields	Relationships
USERS	id PK; email UNIQUE; display_name; status; created_at	One user has many passkeys, sessions, events, and admin actions.
PASSKEY_CREDENTIALS	id PK; user_id FK; credential_id UNIQUE; public_key; sign_count; transports; created_at; last_used_at	Many credentials belong to one user.
ROLES	id PK; name UNIQUE; description	Roles map to users and permissions through bridge tables.
USER_ROLES	user_id FK; role_id FK; assigned_at	Many-to-many bridge between users and roles.
PERMISSIONS	id PK; key UNIQUE; description	Permission definitions.
ROLE_PERMISSIONS	role_id FK; permission_id FK	Many-to-many bridge between roles and permissions.
SESSIONS	id PK; user_id FK; token_hash UNIQUE; created_at; expires_at; revoked_at; auth_time	Many sessions can belong to a user; token hash only, never raw token.
RESOURCES	id PK; name; sensitivity; owner_id FK; status	Protected document/API resource.
ACCESS_POLICIES	id PK; resource_id FK; effect; conditions JSONB; priority; enabled	Many policies may apply to one resource.
SECURITY_EVENTS	id PK; actor_user_id FK NULL; event_type; resource_id FK NULL; outcome; reason_code; correlation_id; occurred_at; event_hash; prev_hash	Append-oriented event history; may be linked to alerts/risk assessments.
ALERTS	id PK; event_id FK; severity; status; acknowledged_by FK NULL; acknowledged_at NULL	Actionable view of selected events.
RISK_ASSESSMENTS	id PK; event_id FK; score; reason_codes JSONB; rule/model_version; created_at	Explainable risk assessment for an event.
ADMIN_ACTIONS	id PK; admin_user_id FK; action; target_type; target_id; created_at	Privileged changes are separately auditable.

Relationship summary

• USERS ↔ ROLES is many-to-many through USER_ROLES.

• ROLES ↔ PERMISSIONS is many-to-many through ROLE_PERMISSIONS.

• USERS 1-to-many PASSKEY_CREDENTIALS, SESSIONS, SECURITY_EVENTS, and ADMIN_ACTIONS (as actor).

• RESOURCES 1-to-many ACCESS_POLICIES; an event may be linked to a resource, alert, and risk assessment.

• Credential ID, session token hash, role name, permission key, and other identity constraints should be unique where appropriate.

5.1 Data Dictionary and Database Rules

Table	Purpose	Important rules
users	Identity profile and account state.	Unique normalized email; constrained status. Avoid password storage if using passkeys exclusively, unless a separately designed password flow is required.
passkey_credentials	Public credential metadata for WebAuthn verification.	Store credential ID/public key and metadata. Never store private key or biometric template.
roles / permissions	Role-based access model.	Unique names/keys; many-to-many bridge tables.
sessions	Server-side session state and revocation.	Hash opaque high-entropy session token; expiry/revoked_at; never log raw tokens.
resources	Protected documents/API resource descriptors.	Authorization enforced server-side on every read/write/action.
access_policies	Rules for roles, resources, freshness, and risk.	Default deny; validate changes; version policies and log edits.
security_events	Append-oriented event records.	Never log passwords, private keys, session tokens, raw biometric data, or unnecessary personal data.
alerts	Actionable view of selected events.	Constrain severity/status; audit acknowledgement.
risk_assessments	Risk score and reason codes.	Record rule/model version; don't make unvalidated AI the sole control.
admin_actions	Privileged action history.	Record actor, action, target, timestamp, and correlation ID; restrict modification/deletion.

Recommended database constraints

• Use foreign keys with explicit ON DELETE behavior; avoid cascading deletion of audit history.

• Index users.email, credential_id, sessions.token_hash, sessions.expires_at, events.occurred_at, events.event_type, and correlation_id.

• Store timestamps in UTC and format local time only in the UI.

• Keep database migrations in source control.

• Make audit retention configurable; restrict exports and redact identifiers where possible.

• For hash-chained events, hash canonicalized event data plus previous hash; periodically anchor checkpoints outside the primary database for stronger tamper evidence.

6. Authentication and Access Workflows

Passkey registration

• User signs in through an approved bootstrap or recovery process and reauthenticates.

• Backend generates registration options with a fresh challenge and expected RP ID; challenge is single-use and short-lived.

• Browser invokes navigator.credentials.create(); authenticator performs local user verification as configured.

• Backend verifies challenge, origin, RP ID, required user verification, and applicable attestation policy.

• Backend stores credential ID and public-key metadata; private key remains with the authenticator/provider.

• Write an audit event and show the user registered authenticator metadata.

Passkey login

• Browser requests authentication options; backend issues a fresh, unpredictable, expiring challenge.

• Authenticator returns a signed assertion after local verification, when required.

• Backend validates challenge, origin, RP ID, signature, credential status, and user-verification requirement.

• Backend creates/rotates a secure session and records successful authentication.

• Protected requests are authorized server-side; high-risk operations require recent authentication or an additional factor.

Access decision pseudocode

```
if account is inactive: DENY
if session is invalid, expired, or revoked: DENY
if user lacks permission for resource/action: DENY
if authentication is too old for action sensitivity: REQUIRE STEP-UP AUTHENTICATION
calculate explainable risk from approved signals
if risk exceeds threshold: DENY or REQUIRE REVIEW according to policy
otherwise: ALLOW and log decision
```

Session lifecycle

Event	Expected behavior
Login success	Rotate session identifier; set Secure, HttpOnly, SameSite cookie; record auth_time and expiry.
Logout	Revoke server-side session and clear cookie.
Recovery	Require strong verification; notify user; revoke/re-evaluate sessions according to policy.
Role change / suspension	Invalidate relevant sessions or immediately re-check authorization; audit the change.
Emergency revoke	Admin revokes session(s), creates audit event, and triggers configured alert.

7. API Design

Prefix routes with /api/v1. Use JSON over HTTPS, consistent errors, request IDs, validation, and server-side authorization. Document exact schemas in OpenAPI.

Method / endpoint	Purpose	Access / notes
POST /auth/register/options	Begin passkey registration	Authenticated/recovery-authorized; rate-limited; one-time challenge.
POST /auth/register/verify	Verify and store passkey	Authenticated; validate complete ceremony.
POST /auth/login/options	Begin authentication	Public but rate-limited; avoid account enumeration.
POST /auth/login/verify	Verify assertion and create session	Strict WebAuthn validation; generic errors.
POST /auth/logout	Revoke current session	Authenticated; CSRF protection where cookie-based.
GET /auth/me	Current user/session metadata	Authenticated; never return secrets.
GET /sessions	List current user's sessions	Authenticated; limit metadata exposed.
DELETE /sessions/{id}	Revoke session	Owner or authorized administrator; audit action.
GET /resources	List permitted resources	Authenticated; filter server-side.
GET /resources/{id}	Read protected resource	Authorization checked on every request.
POST /resources/{id}/actions/{action}	Perform sensitive action	Permission, freshness/step-up checks as required.
GET /admin/events	Search security events	Admin/auditor; pagination and filters.
GET /admin/alerts	List alerts	Authorized role; paginated.
POST /admin/alerts/{id}/acknowledge	Acknowledge alert	Admin permission; audit event.
POST /admin/sessions/{id}/revoke	Emergency revoke	Admin strong auth; audit and alert.
PUT /admin/policies/{id}	Update policy	Restricted admin; validate and audit.

API response and error conventions

• Use 401 for unauthenticated requests and 403 for authenticated but unauthorized requests, without leaking sensitive resource existence where inappropriate.

• Use stable codes such as AUTH_CHALLENGE_EXPIRED, AUTHENTICATOR_INVALID, SESSION_REVOKED, POLICY_DENY, and STEP_UP_REQUIRED.

• Attach a correlation ID to security-relevant responses and events.

• Use pagination, bounded filters, and request-size limits on event/search endpoints.

• Protect cookie-authenticated state-changing requests against CSRF and validate Origin where appropriate.

8. Security, Privacy, and Threat Model

Assume attackers may steal a session, abuse recovery, replay an old challenge, exploit a missing permission check, or compromise an administrator account. Demonstrate prevention and detection in a local or authorized test environment.

Threat	Potential impact	Mitigation / test
Phishing / credential theft	Unauthorized account access	Prefer passkeys; test phishing-resistant flows on supported authenticators; document fallback limits.
Challenge replay	Replay old authentication response	Single-use random challenges with short expiry; reject duplicates/expired challenges.
Origin/RP ID confusion	Assertion accepted by wrong site	Strictly validate origin and RP ID; test wrong-origin responses.
Broken object authorization	User reads another user's document	Server-side ownership/permission checks on every endpoint; negative API tests.
Session theft/fixation	Attacker reuses valid session	Secure cookie flags, rotation, expiry, revocation, HTTPS; no tokens in URLs.
Brute-force / denial of service	Login endpoint abuse	Rate limits, bounded work, alerting, safe error messages.
Admin account compromise	Policies/users/sessions manipulated	Strong auth, least privilege, step-up for critical actions, audit changes.
Audit tampering	Incident history altered	Append-oriented permissions, hash chain, external checkpoint, integrity tests.
Sensitive data leakage	Credentials or personal data exposed	Redaction, least-privilege DB access, secrets management, minimal retention.
Unsafe AI decision	False blocks or missed attacks	Start with explainable rules; measure errors; human review for high-impact actions.
Biometric overclaim	False assurance or inaccessible login	Describe device support accurately; provide accessible fallback and recovery.

Privacy requirements

- Do not collect or store raw fingerprints, face images, or iris templates for WebAuthn passkey authentication.
- Store only public credential material and minimum account/security metadata needed.
- Provide a clear privacy notice, retention rules, account deletion policy, and support process.
- Avoid retaining IP/device details longer than needed; document purpose and retention.
- Any separate camera-based biometric feature needs its own privacy/security assessment and appropriate consent.

Illustrative risk scoring (demo only)

Signal	Illustrative points	Action idea
Repeated failed attempts	+25	Rate-limit and request step-up verification.
New authenticator/device context	+10	Step-up for sensitive resource; do not assume compromise automatically.
Sensitive resource + stale authentication	+20	Require fresh authentication.
Revoked session / inactive account	Hard deny	Reject regardless of score.
0–19 / 20–39 / 40+	Low / elevated / high	Allow / step-up / deny or review per documented policy.

9. UI Screens and User Stories

Screen	Main elements	Acceptance notes
Public landing / login	Product summary, passkey sign-in, accessible fallback, help/recovery link	No protected data before authentication.
Passkey enrollment	Browser prompt, authenticator label, success/failure, remove credential	Only account owner/authorized admin manages credentials.
Employee home	Permitted resources, recent sign-ins, active sessions	Only backend-permitted resources returned.
Protected document view	Metadata, content, sensitivity label, last-auth indicator	Direct URL/API access without permission is denied.
Security dashboard	Event counts, trends, alerts, risk reasons, filters	Admin/auditor authorization enforced.
Event detail	Timestamp, actor, event, outcome, reason, correlation ID	No raw tokens, secrets, or biometric data.
Policy management	Role/resource/action rules, enabled state, simulation	Changes require permission, validation, confirmation, audit.
Session management	Current sessions, last activity, revoke action	Revocation invalidates session server-side.
Recovery / help	Secure recovery process and support guidance	Recovery must not bypass the security model.

Example user stories

- As an employee, I can register a passkey to authenticate with a supported device authenticator.
- As an employee, I can view only resources permitted by my role.
- As an administrator, I can revoke a compromised session and see an audit record.
- As an auditor, I can review events without changing policies or accounts.
- As a security administrator, I can see why a request was flagged and which policy action occurred.
- As a user without biometric-capable hardware, I can use a supported alternative authenticator or recovery process.

10. Test Plan and Acceptance Criteria

Run tests only against your own application and systems you are authorized to test. Use automated API tests plus a manual cross-device browser matrix.

Test ID	Scenario	Expected result
T-01	Register passkey on supported browser/device	Credential verified and public-key metadata stored; no biometric image stored.
T-02	Authenticate with valid assertion	Session created with secure settings; success event logged.
T-03	Submit wrong/expired challenge	Authentication rejected; no session; safe event logged.
T-04	Wrong origin/RP ID/signature	Authentication rejected.
T-05	Access resource without session	401 or safe sign-in redirect; no content disclosed.
T-06	Employee accesses admin endpoint	403; no privileged action; event logged.
T-07	Employee guesses another resource ID	Server denies unauthorized object access.
T-08	Use revoked or expired session	Request rejected within documented revocation target.
T-09	Repeated failed authentication	Rate limit/protective response and configured alert.
T-10	Admin revokes a session	Session no longer accesses protected APIs; admin action audited.
T-11	Modify/delete event in test database	Integrity verification detects chain inconsistency; document limitations.
T-12	Use browser/device without platform biometrics	Supported alternative offered; no false promise of biometric support.
T-13	Stale authentication attempts sensitive action	Step-up authentication required.
T-14	Inspect logs and API responses	No private keys, raw tokens, passwords, or biometric data leaked.

Definition of done

- Core user flows work in a clean environment using documented setup steps.
- No critical/high severity issue remains in the agreed MVP test scope; limitations are disclosed.
- Backend authorization tests pass; demo shows both allowed and denied access.
- Architecture, ERD, API spec, threat model, test report, and demo script are committed to GitHub.
- README labels the project as a prototype and does not claim third-party certification.

11. Implementation Roadmap and Repository Structure

Phase	Work	Exit criterion
0 — Planning	Finalize requirements, threat model, ERD, API contract, wireframes.	Reviewed scope and backlog.
1 — Foundation	Repo, Docker Compose, FastAPI, React, PostgreSQL migrations, CI.	Clean install and health checks work.
2 — Identity	WebAuthn registration/login, session lifecycle, recovery design.	Passkey flows pass positive/negative tests.
3 — Authorization	Roles, permissions, resources, policy checks, step-up freshness.	Unauthorized direct API requests are denied.
4 — Audit / risk	Event schema, dashboard, risk reason codes, revocation.	Events and admin actions visible/testable.
5 — Hardening	Threat-driven tests, dependency scan, rate limits, security headers, backups.	Documented test report and remediation.
6 — Showcase	Seed data, README, diagrams, screenshots, short demo video.	Reviewer can run and evaluate project.

Suggested repository structure

```
sentinel-x/
■■■ apps/
■ ■■■ web/ # React + TypeScript frontend
■ ■■■ api/ # FastAPI application
■■■ packages/
■ ■■■ contracts/ # Shared API schemas (optional)
■■■ infra/
■ ■■■ docker-compose.yml
■ ■■■ reverse-proxy/ # HTTPS configuration
■■■ database/
■ ■■■ migrations/
■ ■■■ seed/ # Synthetic demo data
■■■ docs/ # Requirements, architecture, DFD, ERD, API, threat model, tests
■■■ tests/
■ ■■■ api/
■ ■■■ security/
■ ■■■ e2e/
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

Suggested GitHub epics

- Authentication and passkey ceremonies.
- RBAC, resource permissions, and step-up policy.
- Session security and revocation.
- Audit event pipeline and tamper-evidence verification.
- Security dashboard and alert workflows.
- Threat-driven automated tests and documentation.

12. Deployment, Demo Plan, Risks, and References

Deployment baseline

• Local development: Docker Compose for API, PostgreSQL, optional Redis; run frontend separately or in a container.

• Hosted demo: HTTPS with configured relying-party ID/origin; WebAuthn settings must match the real domain.

• Configuration: .env.example contains placeholders only; never commit secrets, private keys, production credentials, or real user data.

• Database: migrations, least-privilege application DB user, backup/restore instructions, and synthetic seed data.

• Observability: structured logs with request/correlation IDs, safe redaction, health endpoints, and retention rules.

• CI: lint, type checks, unit tests, API tests, dependency/security checks, and build verification.

Two-minute hackathon demo

Time	Demonstration
0:00–0:20	Explain the problem and show the protected resource portal.
0:20–0:45	Use a pre-enrolled passkey on a supported device; complete sign-in.
0:45–1:05	Open an authorized resource and show backend access decision and audit event.
1:05–1:25	Switch to a lower-privilege demo account and attempt the same resource; show denial.
1:25–1:45	Trigger a safe simulated suspicious event; show risk reasons and alert.
1:45–2:00	Revoke a demo session; show protected requests fail and admin action is audited.

Main risks and mitigations

Risk	Mitigation
Hardware/browser differences	Publish tested compatibility matrix; use passkeys and supported fallback; do not promise all biometrics everywhere.
Scope too large	Ship complete MVP before camera-based face or iris features.
False positive risk scores	Explain signals; tune with synthetic cases; use review/step-up rather than blind AI decisions.
Audit changed by DB admin	Hash chains detect many changes but are not enough alone; use external checkpoints and restricted append path.
Demo leaks secrets/data	Synthetic accounts/data; environment secrets; redact logs; review repository before publishing.

Documentation and standards to consult

• NIST SP 800-207, Zero Trust Architecture: https://csrc.nist.gov/pubs/sp/800/207/final

• W3C Web Authentication (WebAuthn): https://www.w3.org/TR/webauthn-3/

• FIDO Alliance, Passkeys: https://fidoalliance.org/passkeys/

• OWASP Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

• OWASP Authorization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

• OWASP Logging Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

Final recommendation

Build the first release around standards-based passkeys, server-side authorization, a protected resource portal, session revocation, and explainable security events. Treat face/iris recognition as an optional separate integration after the core gateway is secure and working. Strong portfolio evidence is a reproducible repository, clear diagrams, automated negative tests, and a live demo proving both access granted and access denied.